GUARDIA
CYBERSECURITY SCHOOL

CAHIER DES CHARGES

Implémentation d'un coffre-fort numérique sécurisé

Gaming Campus - © Active Learning by Quest Education Group
Document Interne 2025-2026

INTRODUCTION

Nom du projet : Implémentation d'un coffre-fort numérique sécurisé
N° du Projet : 4
Nom d'UE : DevSecOps
N° d'UE : 7
Date : Du 20/04/2026 au 29/04/2026
Classe : Bachelor 2e année

Modalités

Groupes de 2 étudiants à former avant le début du projet

Contexte et Description du Projet

Contexte

À l'ère du *« tout numérique »*, notre quotidien repose sur la production, l'utilisation, le stockage, ou encore, l'échange d'informations. Qu'il s'agisse de données médicales, de transactions bancaires, ou de démarches administratives en ligne, ces données sont potentiellement sensibles et nécessitent d'être protégées. En effet, toute donnée pouvant porter atteinte à une personne ou à une organisation en cas de divulgation est considérée comme sensible.

Détail du projet

Objectifs

Ce projet a pour principaux objectifs d'appréhender les notions de chiffrement (symétrique et asymétrique) et de hachage ainsi que les principaux algorithmes actuels correspondant afin de les mettre en œuvre au moyen du langage Python.

Parmi les objectifs secondaires, les étudiants seront amenés à se perfectionner du point de vue algorithmique et de la programmation en langage Python, tout en adoptant les *« bonnes pratiques »* en termes de développement logiciel (conventions, organisation du code, gestion de versions avec Git, tests, autres).

Spécifications - Version minimale

Gaming Campus - © Active Learning by Quest Education Group
Document Interne 2025-2026

« GuardiaBox » est une application qui permet de chiffrer, stocker et déchiffrer des fichiers ou des messages de manière sécurisée.
Cette application repose sur certains principes de la programmation algorithmique sécurisée à travers :

- L'usage d'algorithmes de chiffrement.
- La gestion correcte des clés et des mots de passe.
- La validation et la sécurisation des entrées et des sorties.
- La résistance à certaines vulnérabilités courantes (ex. : attaque par dictionnaire, mauvaises pratiques de hachage).

Les fonctionnalités minimales attendues, via une interface utilisateur en mode console (menu de l'application) sont les suivantes :

1. Chiffrer un fichier ou un message.
2. Déchiffrer un fichier ou un message.
3. Quitter.

L'architecture logicielle (organisation des différents composants de l'application du point de vue du système de fichiers) doit respecter la structure ci-dessous :

```
guardiabox/
├── fileio/
├── security/
├── tests/
├── ui/
└── main.py
```

- **guardiabox :** racine du projet.
- **fileio :** contient le(s) module(s) destiné(s) à la manipulation des fichiers (ex. : lecture, écriture).
- **security :** contient le(s) module(s) destiné(s) à chiffrer, hacher, gérer des clés, assurer l'intégrité des données, etc.
- **tests :** contient le(s) module(s) destiné(s) à tester l'application.
- **ui :** contient le(s) module(s) destiné(s) à mettre en œuvre l'interface utilisateur en mode console.
- **main.py :** point d'entrée principal de l'application.

Du point de vue de l'implémentation, quelques pistes concernant les éléments à mettre en œuvre :

- Chiffrement symétrique (AES-GCM) :
  - L'utilisateur fournit un message ou un fichier (nom de fichier).
  - S'il s'agit d'un message, un fichier texte contenant le message est créé.
  - Il saisit un **mot de passe**.
  - Une **clé dérivée** est générée via PBKDF2 (sel aléatoire + itérations) afin de chiffrer le fichier.
  - Le fichier chiffré contient : `salt` + `nonce` + `ciphertext` + `tag`.

    - Le nom du fichier chiffré reprend le nom d'origine auquel l'extension `.crypt` est ajoutée.
- Déchiffrement sécurisé :
    - L'utilisateur saisit le même **mot de passe**.
    - Reconstitution de la **clé** depuis le **mot de passe** et le **sel** via la lecture du fichier `.crypt`.
    - Vérification de l'**intégrité** du fichier via le tag AES-GCM.
    - Affichage du message en clair ou création du fichier déchiffré (ajout d'une extension `.decrypt`).
- Validation des entrées (liste non exhaustive) :
    - Dans le cas d'un fichier, s'assurer que le fichier existe.
    - S'assurer que le mot de passe est suffisamment sécurisé (robustesse, entropie).
    - Empêcher les injections de chemin (ex. : `"../../"`).
    - Etc.
- Exemples de tests unitaires (Pytest) recommandés :
    - Vérifier que le chiffrement, suivi du déchiffrement, rendent bien la donnée initiale.
    - Vérifier qu'un mauvais mot de passe ne permet pas le déchiffrement.
    - Autres.

Spécifications - Version étendue

Selon le temps disponible, quelques pistes d'améliorations envisageables :

- Gestion multi-utilisateurs avec une base de données locale sécurisée (SQLite).
- Historique des opérations dans la base de données locale (SQLite).
- Création d'une interface graphique (PyQt6).
- Chiffrement asymétrique (ex. : RSA) pour partager un fichier chiffré entre utilisateurs.
- Suppression sécurisée (effacement irrécupérable d'un fichier non chiffré).

Progression

Semaine 1 - Remise à niveau, planification, préparation de l'environnement de développement :

- Création d'un *kanban* sous Trello.
- Remise à niveau en algorithmique et programmation Python.
- Remise à niveau sur le thème du *versioning* (Git).
- Perfectionnement sur les concepts-clés de la cryptologie.
- Mise en place de la base de travail.

- Définition des tâches à réaliser et répartition de ces dernières au sein des membres de l'équipe de projet.

Semaine 2 - Planification, implémentation du coffre-fort numérique sécurisé, présentation du travail

- Mise à jour du *kanban* sous Trello.
- Implémentation du projet en respectant les spécifications présentées en amont (section *« Spécifications - Version minimale »*).
- Selon le temps disponible, amélioration du projet (section *« Spécifications - Version étendue »*).
- Présentation du travail réalisé.

OBJECTIFS PÉDAGOGIQUES ET PROFESSIONNELS DU PROJET

A l'issue de ce projet, les étudiants seront capables de (Veuillez trouver les verbes actifs et mesurables dans cette infographie) :

Savoirs

- Comprendre les fondamentaux de l'algorithmique.
- Maîtriser les bases du langage Python.
- Comprendre les concepts de *packages* et de modules (Python).
- Comprendre les bases de la programmation orientée objet.
- Définir les opérations d'entrée/sortie sur un fichier texte.
- Définir ce qu'est une structure de données.
- Comprendre les structures de données en Python (ex. : listes, tuples, dictionnaires).
- Définir la notion de SGBDR (Système de Gestion de Base de Données Relationnelle).
- Comprendre le principe de la gestion de version (Git).
- Prendre connaissance des *« bonnes pratiques »* et conventions (ex. : PEP 8).
- Comprendre le concept et l'intérêt des tests unitaires.
- Définir ce qu'est la cryptographie.
- Distinguer chiffrement et hachage.
- Comprendre les notions de chiffrement symétriques et asymétriques.
- Comprendre les concepts avancés en lien avec le hachage (ex. : salage, force brute).

Savoir-faire / Compétences

- Interpréter un cahier des charges.
- Lire une documentation technique.
- Élaborer un algorithme.
- Traduire un algorithme dans un langage de programmation.
- Configurer un environnement de développement.
- Utiliser un éditeur de code.
- Écrire et exécuter un script en langage Python.
- Établir un choix d'algorithme cryptographique sur la base de critères objectifs.
- Utiliser des *packages* Python adaptés (ex. : `cryptography`, `hashlib`, `hmac`).
- Mettre en œuvre les *« bonnes pratiques »* du point de vue de la programmation.
- Créer une base de données relationnelle SQLite.
- Exploiter une base de données grâce au langage SQL (CRUD).
- Interagir avec une base de données SQLite en langage Python.
- Présenter son travail.

Savoir-être

- Interagir avec son *manager* de façon pertinente.
- Travailler en équipe.
- Collaborer avec efficacité.
- Planifier le bon déroulement du projet.
- Respecter les délais.
- Gérer son temps.
- S'adapter aux imprévus.
- Faire preuve de curiosité.
- Faire preuve de persévérance.
- Faire preuve d'initiative.
- Travailler en autonomie.
- Adopter un point de vue global (prise de recul).
- Adopter un point de vue critique.
- Évaluer sa propre contribution.
- Reconnaître la contribution des autres.

Ressources

1. **A savoir/à apprendre** (Ce travail pourra être évalué / noté en début de projet) :
    - Intro to computer science - Python | Khan Academy
    - Genepy — Exercices Python
    - Introduction au SQL : interroger et gérer des données | Khan Academy
    - SQLite Tutorial - An Easy Way to Master SQLite Fast
    - Git - Learn
    - Le PEP 8 en résumé !
2. **A lire/à consulter** :
    - Cryptographie — Wikipédia
    - Chiffrement — Wikipédia
    - Fonction de hachage — Wikipédia
    - Python
    - PyPI · The Python Package Index
    - Python Cheat Sheet – Real Python
    - sqlite3 — DB-API 2.0 interface for SQLite databases — Python 3.14.0 documentation
    - SQLite Home Page
    - Cryptographic Services — Python 3.14.0 documentation
    - cryptography · PyPI

3. **A installer/pré-requis techniques** :
    - Visual Studio Code - Code Editing. Redefined
    - Python
    - Git
    - Manage Your Team's Projects From Anywhere | Trello

Travail préparatoire

1. Pratiquer le langage Python via la Khan Academy et/ou Genepy (voir liens ci-dessus).
2. S'assurer de la disponibilité et du bon fonctionnement des logiciels et applications nécessaires (VS Code, Python, Git).

INTERVENANT

Nom : Sylvain Labasse
Titre : Consultant / Développeur
Email : labasse@gmail.com
Site web : https://syllab.com
LinkedIn : https://www.linkedin.com/in/labasse

BIOGRAPHIE

Passionné de création, je programme par plaisir.
A 11 ans, je gagne mon premier micro-ordinateur à un concours de dessin dont je sature vite les 2ko de RAM avec mes programmes en basic. Dix ans plus tard, étudiant en informatique industrielle, je crée sous windows 3.11 des vocodeurs et pilote des caméras de surveillance. A l'université, je co-écris un shareware pour jouer en réseau à Civilization II de Microprose, ce qui m'ouvre les portes du plus grand studio de développement indépendant de jeux de l'époque. A l'explosion de la bulle Internet, je deviens freelance. Tout en enseignant en école d'informatique, je réalise depuis 2004 des missions en entreprise. Notamment en 2014, je fais une mission dans l'équipe engine d'Asobo Studio pour le portage de The Crew d'UbiSoft sur Xbox 360 et je contribue à Microsoft HoloTour sur HoloLens.

ROADMAP

DESCRIPTION

Veuillez trouver ci-dessous une description des livrables attendus ainsi que les dates d'échéance associées. Il est essentiel de respecter les échéances suivantes pour assurer une progression harmonieuse et structurée du projet. Chaque livrable représente une étape importante dans le processus de réalisation et permet d'évaluer l'avancement du travail. Les dates limites fixées doivent être rigoureusement respectées afin de garantir une évaluation équitable et de permettre un feedback constructif en temps opportun.

Jalon	Livrables attendus	Date limite	Moyens / formats
1	Kanban Git Base de travail	Semaine 1 Jour 5	*Screenshot* du tableau de bord sous Trello *Screenshot* du dépôt GitHub *Screenshot* de l'arborescence du projet
2	Bilan intermédiaire Pré-version du support de présentation	Semaine 2 Jour 3	Entretien oral avec chaque groupe (présentation de l'avancement du travail) Lien de partage au document Google Slides (avec rôle « *Éditeur* »)
3	Bilan du projet	Semaine 2 Jour 5	Présentation orale collective par équipe (10 minutes) Difficultés rencontrées Bilan des acquis Perspectives et améliorations de l'application « *GuardiaBox* »

Gaming Campus - © Active Learning by Quest Education Group
Document Interne 2025-2026

EVALUATION

L'évaluation est conçue pour être holistique, prenant en compte non seulement le produit final, mais aussi le processus, les compétences acquises et les attitudes démontrées tout au long du projet. Les compétences sont classées en trois catégories principales : "savoirs" (connaissances théoriques), "savoir-faire" (compétences pratiques), et "savoir-être" (compétences interpersonnelles et attitudes).

Système de Notation

Savoirs (Connaissances)

- **Compréhension théorique** : Évaluée soit en amont du projet pendant la semaine théorie soit lors de la soutenance et restitution du projet. Cela permet de mesurer la compréhension des concepts fondamentaux et des connaissances liées au projet des étudiants.

Savoir-faire (Compétences)

- **Compétences techniques et application** : Évaluées à travers la soumission finale du projet. Cela inclut la qualité, la fonctionnalité, et la précision technique du travail produit.
- **Gestion de projet** : Évaluée en fonction de l'organisation, du respect des échéances, et de l'utilisation efficace des ressources. Cela peut être évalué à travers la documentation du projet et les journaux de processus.

Savoir-être (Attitudes/Compétences interpersonnelles)

- **Travail d'équipe et collaboration** : Évalués à travers des évaluations par les pairs et les membres du groupe. Les critères incluent la communication, la coopération, et la contribution aux tâches du groupe.

- **Autonomie et initiative** : Évaluées en fonction des contributions individuelles, de la capacité à travailler de manière autonome, et de la résolution proactive des problèmes.
- **Soutenance** : Évaluées lors de la présentation finale du projet. Les critères incluent la clarté, la cohérence, et la capacité à articuler et défendre les résultats du projet.

Gaming Campus - © Active Learning by Quest Education Group
Document Interne 2025-2026

GRILLE D'ÉVALUATION

Dans la grille d'évaluation qui suit, chaque groupe de compétences doit être évalué sur une échelle de valeurs établie de 0 à 3 :

0. *Non acquis*
1. *Partiellement acquis*
2. *Acquis*
3. *Acquis et dépassé*

#	Compétences	Niveau d'acquisition (de 0 à 3)
	1 - Savoirs	
01	Comprendre les fondamentaux de l'algorithmique.	
02	Maîtriser les bases du langage Python. Comprendre les concepts de *packages* et de modules (Python). Comprendre les bases de la programmation orientée objet.	
03	Définir les opérations d'entrée/sortie sur un fichier texte. Définir ce qu'est une structure de données. Comprendre les structures de données en Python (ex. : listes, tuples, dictionnaires).	
04	Définir la notion de SGBDR (Système de Gestion de Base de Données Relationnelle).	
05	Comprendre le principe de la gestion de version (Git). Prendre connaissance des *« bonnes pratiques »* et conventions (ex. : PEP 8). Comprendre le concept et l'intérêt des tests unitaires.	
06	Définir ce qu'est la cryptographie. Distinguer chiffrement et hachage.	
07	Comprendre les notions de chiffrement symétriques et asymétriques.	

Gaming Campus - © Active Learning by Quest Education Group
Document Interne 2025-2026

	Comprendre les concepts avancés en lien avec le hachage (ex. : salage, force brute).	
	2 - Savoir-faire	
08	Interpréter un cahier des charges. Lire une documentation technique.	
09	Élaborer un algorithme. Traduire un algorithme dans un langage de programmation.	
10	Configurer un environnement de développement. Utiliser un éditeur de code. Écrire et exécuter un script en langage Python.	
11	Établir un choix d'algorithme cryptographique sur la base de critères objectifs. Utiliser des packages Python adaptés (ex. : `cryptography`, `hashlib`, `hmac`).	
12	Mettre en œuvre les *« bonnes pratiques »* du point de vue de la programmation.	
13	Créer une base de données relationnelle SQLite. Exploiter une base de données grâce au langage SQL (CRUD).	
14	Interagir avec une base de données SQLite en langage Python.	
15	Présenter son travail.	
	3 - Savoir-être	
16	Interagir avec son *manager* de façon pertinente. Travailler en équipe. Collaborer avec efficacité.	
17	Planifier le bon déroulement du projet. Respecter les délais. Gérer son temps. S'adapter aux imprévus.	
18	Faire preuve de curiosité. Faire preuve de persévérance. Faire preuve d'initiative. Travailler en autonomie.	
19	Adopter un point de vue global (prise de recul). Adopter un point de vue critique.	
20	Évaluer sa propre contribution.	

	Reconnaître la contribution des autres.	
	Total	
	Somme totale des niveaux d'acquisition	
	Moyenne finale (/20) *(voir détail du calcul ci-dessous)*	

La **moyenne finale (/20)** est calculée de la manière suivante :

- 20 * somme totale des niveaux d'acquisition / (nombre d'items * valeur maximale du niveau d'acquisition) ;
- soit, 20 * somme totale des niveaux d'acquisition / 60.

EXAMEN INDIVIDUEL DE FIN DE PÉRIODE

Notez ici les éléments relatifs au partiel : acquis à valider / modalités envisagées :

- Bilan des acquis par le biais d'un partiel de 1h reposant sur des notions théoriques ainsi que sur la mise en œuvre d'algorithmes de hachage et de chiffrement en langage Python.